

Coverage Matrix

Every topic the لائحة declares, and where this repository implements it

Dr. Haitham A. El-Ghareeb

Fall 2026

Contents

The matrix	1
Topic by topic, in more detail	3
Deliberately outside the declared content	4
Under the 2026 bylaw, from next year	4

This page exists so that coverage is **checkable**, not asserted. One row per topic declared by a bylaw, the verbatim phrase that declares it, the page it is on, the module that implements it, the test file that grades it, and the week it is taught.

If a row is wrong or a topic is missing, that is a bug in the course and I want to hear about it.

The two declarations being covered

Bylaw	Programs	Spec page
2013 / 2014 (<i>identical text</i>)	Software Engineering, Bio / Medical Informatics	p. 38 / p. 35
2020	Artificial Intelligence	p. 44

This course teaches the **union** of the two, so nobody is short-changed by which bylaw they happen to fall under.

The matrix

#	Declared topic	2013/14	2020	Module	Test	Week
1	Abstract data types	—	"abstract data types"	(<i>concept</i>) dsa/__init__.py	—	1

#	Declared topic	2013/14	2020	Module	Test	Week
2	Algorithmic analysis / complexity	"the basics of algorithmic analysis"	"analyzing and managing the complexity"	viz/complexity.py dsa/array_ops.py	test_array_ops.py	
3	Recursion	"Topics include recursion"	—	dsa/recursion.py	test_recursion.py	
4	OOP philosophy	"the underlying philosophy of object-oriented programming"	—	(concept) — every class in dsa/	—	1, 4–14
5	Arrays	—	"arrays"	dsa/array.py	tests/test_array.py, (given), tests/test_array_ops.py, dsa/array_ops.py, tests/test_dynamic_array.py dsa/dynamic_array.py	
6	Lists / linked lists	"linked lists"	"lists, linked lists"	dsa/linked_lists.py	tests/test_linked_list.py	
7	Stacks	"stacks"	"stacks"	dsa/stack.py	tests/test_stack_queue.py	
8	Queues	"queues"	"queues"	dsa/queue.py	tests/test_stack_queue.py	
9	Searching	—	"searching"	dsa/searching.py	tests/test_searching.py	
10	Sorting	—	"sorting"	dsa/sorting.py	tests/test_sorting.py	
11	List manipulation	—	"algorithms ... used for list manipulation"	dsa/linked_lists.py dsa/sorting.py	tests/test_linked_list.py	
12	Trees	"trees"	"trees"	dsa/tree.py	tests/test_tree.py	
13	Tree traversals	—	"tree traversals"	dsa/tree.py	tests/test_tree.py	
14	Heaps	—	"heaps"	dsa/heap.py	tests/test_heap.py	
15	Priority queues	—	"priority queues"	dsa/heap.py	tests/test_heap.py	
16	Hash tables	"hash tables"	"hash tables"	dsa/hashmap.py	tests/test_hashmap.py	
17	Graphs	"graphs"	"graphs"	dsa/graph.py	tests/test_graph.py	
18	Graph searches	—	"graph searches"	dsa/graph.py	tests/test_graph.py	

#	Declared topic	2013/14	2020	Module	Test	Week
19	Principles of language translation	"an introduction to the principles of language translation"	—	dsa/translation.py	test_translation.py	15

Nineteen declared topics, nineteen covered.

Topic by topic, in more detail

1 — Abstract data types. Declared only by the 2020 text, but it is the idea the whole course rests on, so it is taught in Week 1 to everyone: an ADT is a *contract* (what operations exist and what they cost); a data structure is one *arrangement of memory* that keeps that contract. One ADT, many structures.

2 — Algorithmic analysis. Big-O and friends, then `viz.complexity.measure()` so you check your reasoning against a clock instead of trusting it. Note that neither MATH012 *Discrete Structures* (2013 p. 33, 2014 p. 30) nor CS012 teaches asymptotics, so for SWE and Bio students this is first contact and is taught from zero. AI students have seen "asymptotic notations" in MT1001 (2020, p. 40).

3 — Recursion. Declared by the 2013/2014 bylaws and taught **nowhere before this course in any of the three programs**. Week 3, before it is needed by merge sort, quicksort, tree traversals and the parser.

4 — OOP philosophy. Declared by the 2013/2014 bylaws — which matters, because for Bio students this course is their **first exposure to object orientation** (CS012 in the 2014 bylaw is structured programming, p. 33). It is not taught as a separate unit but as the medium: every structure in `dsa/` is a class that owns its data and exposes an interface, and the reason is discussed in Week 1. AI students arrive having already taken CS1002 (2020, p. 42) and can treat it as revision.

5–11 — The linear structures. Arrays, linked lists, stacks, queues, searching and sorting. Already scaffolded before this pass; unchanged.

12–13 — Trees and traversals. Binary search trees with all three delete cases, plus in-order, pre-order, post-order and level-order. Level-order needs a queue rather than the call stack, which is the point of teaching it after Week 7.

14–15 — Heaps and priority queues. Declared by the 2020 text. Taught to everyone: the array *is* the tree, `heapify` in $O(n)$, and a priority queue built on top rather than reimplemented.

16 — Hash tables. Chaining and open addressing with tombstones, both scaffolded, and the honest discussion of what " $O(1)$ average" does and does not promise.

17–18 — Graphs and graph searches. Adjacency list and adjacency matrix, paired so the space/time trade-off is measured rather than asserted. BFS, DFS both ways, shortest path by edge count, components, cycles, topological sort.

19 — Principles of language translation. The phrase in the 2013/2014 bylaws is not decoration: **SWE141 Software Construction** takes this course as a prerequisite and covers “BNF and basic theory of grammars and parsing... formal languages” (2013, p. 42). Week 15 builds the small version — tokenizer, recursive-descent parser, AST, evaluator — and needs a stack, recursion and a tree to do it, which makes it the natural capstone.

Deliberately outside the declared content

Topic	Module	Why it is here anyway
Dynamic programming	<code>dsa/dynamic_programming.py</code>	Declared by no bylaw for this course. In AI it belongs to AI3001 (2020, p. 48); in SWE and Bio there is no later algorithms course, so this file is their only structured route to it. Enrichment, not examinable .

And what this course does **not** reach, named so you know the words exist: balanced trees (AVL, red-black), weighted shortest paths (Dijkstra), minimum spanning trees, and NP-completeness. None is declared by any of the three bylaws for this course. AI students meet all four in AI3001.

Under the 2026 bylaw, from next year

CS201 (FCIS Bylaw 2026, p. 138) declares: “arrays, lists, linked lists, stacks, queues, hash tables, heaps, priority queues, graphs, and trees... list manipulation, graph searches, sorting, searching, and tree traversals... analyzing and managing the complexity”.

That is **rows 1–2, 5–18** of the matrix above — the 2020 list, essentially unchanged. It drops “principles of language translation” (row 19) and no longer names recursion or OOP philosophy explicitly (rows 3–4).

So this repository already covers the 2026 course in full, with three topics to spare. Next year’s switch is a documentation edit, not a rewrite, which is the main reason this page exists.

Sources: *regulations/Program-SoftwareEngineering-2013.pdf* p. 38 · *regulations/Program-MedicalInformatics-2014.pdf* p. 35 · *regulations/Program-ArtificialIntelligence-2020.pdf* p. 44 · *regulations/FCIS-ByLaw-2026.pdf* p. 138.